Warp Drive: Technical Architecture

Warp Drive delivers enterprise-grade, verifiable off-chain compute to the Stellar ecosystem.

Warp Drive provides a trusted compute layer that turns arbitrary off-chain data and processes into provably correct on-chain actions, meeting the security, auditability, and performance expectations of Stellar's enterprise audience. The architecture leverages Soroban's smart-contract capabilities, modern WASI execution, and flexible operator governance to create a scalable, composable platform ready for real-world financial and compliance use cases.

Run bots, oracles, automation, and more on Stellar with Eigenlayer-grade security.

Contents

1. Main Components
2. System Diagram
3. Use Cases
4. Implementation Roadmap
    1. Detailed Design and Infrastructure
    2. Soroban Security Contracts
    3. End-to-End Workflow: DeFi Automation
    4. Real World Usage: Protocols on Mainnet
5. Operational Flow
6. Component Deep Dive
7. Conclusion

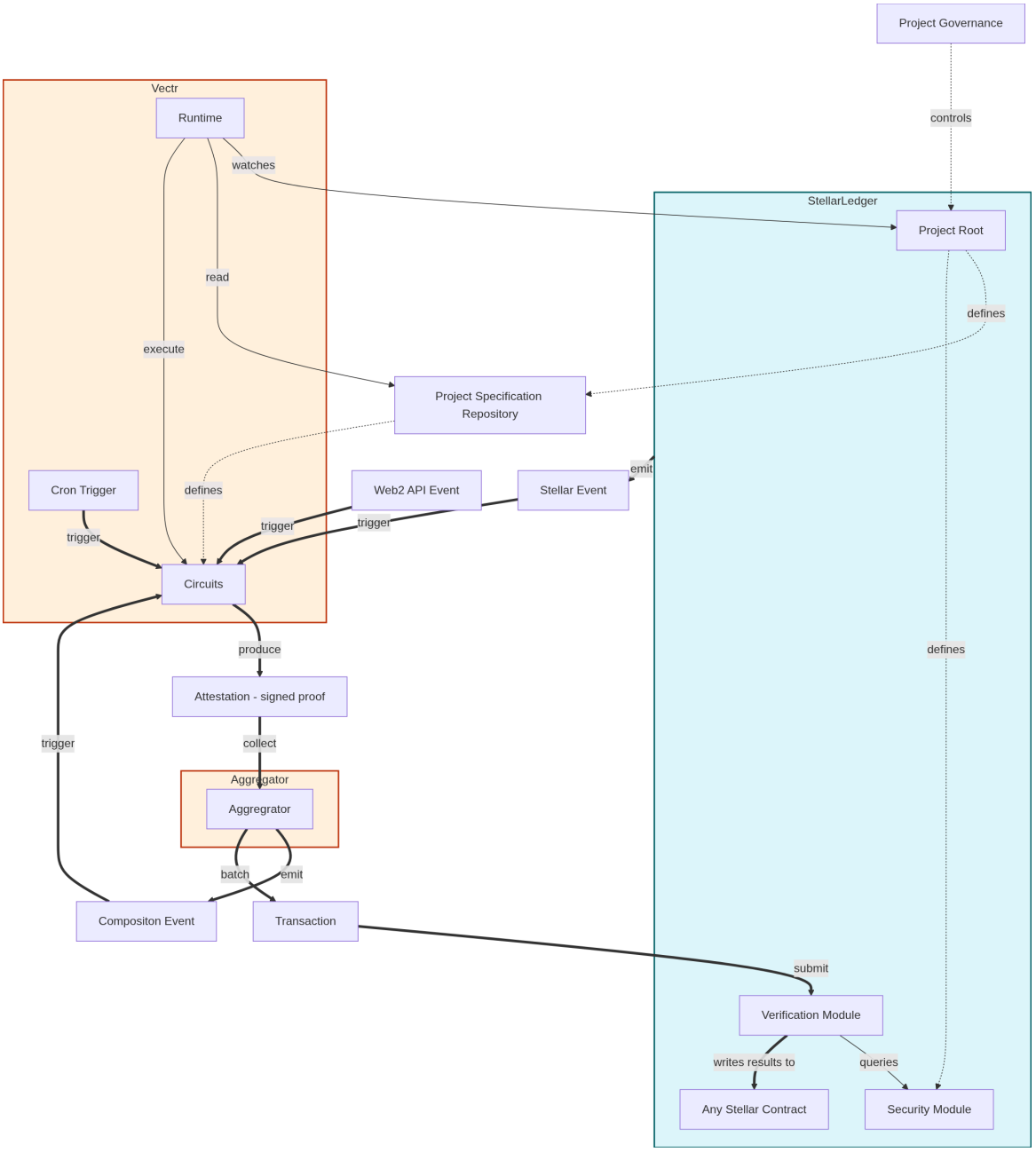
Addendums:

1. Performance
2. Trust Model

Main Components

- **Projects -** Warp Drive is created around the concept of independent "Projects". Each Project has its own governance model and is sandboxed from others. They interact only via contracts on the Stellar blockchain, but their governance and coordination is completely separate
- **Project Specification Repository** – Each *Project* is rooted in a governance-controlled Soroban contract that points to the current specification. The off-chain repository contains immutable JSON/YAML files describing each circuit and all required resources, as well as the security and verification modules that process the outputs, all cryptographically verifiable.
- **Verifiable Computers (Vectrs)** – A decentralized pool of trusted execution nodes that can pull the *Project Specification*, execute any contained *Circuits* and, and produce a signed attestation.
- **Aggregator -** A computer that collects attestations from the various Vectrs and submits them all as one batch on-chain when there are sufficient. This cannot forge signatures but could censor them (or crash). Multiple (decentralized) aggregators could be used to remove any censorship concerns.
- **Circuit Definition** – A *Project Specification* defines multiple off-chain Circuits. Each Circuit defines the event that triggers execution (input), the WASI code run by Vectrs (transformation), and where to submit the resulting attestation (output). Inputs could be based on periodic cron jobs, on-chain event triggers, or web2 APIs. These active circuits are the core value of Warp Drive (the resulting infrastructure is focused on providing cryptographic security to run these off-chain)
- **Verification Module** – A Soroban smart contract that collects *Vectr* attestations produced by the *Circuits*, verifies their cryptographic proofs, and takes and action on the Stellar ledger based on this result.
- **Security Module -** This defines the rules of which *Vectrs* can produce attestations that will be considered valid by this project's *Verification Modules*. This could be PoA for a fixed set of trusted operators. It could be PoS with Eigenlayer(-like) restaking securing a permissionless operator set. Or it could be any algorithm you can define that produces a set of weights assigned to public keys of *Vectrs* that your project trusts.
- **Composable Workflows** – Multiple circuits in the same *Project* can be chained together, allowing complex business processes to be built from reusable building blocks without exposing multichain concerns. Different Projects can be composed.

System Diagram



Use Cases

DeFi Automation

Off-chain limit order. As this is not run on a single bot machine, the high security level means this can be used for balancing or liquidating protocol treasuries.

1. Deploy a Limit Order contract - users can deposit tokens in it and specify what their desired swap is.
2. Circuit by Swap Events on one or more Soroban DEX pools
3. Process event by checking spot price and comparing to pending limit orders (off-chain - not compute limited)
4. If price now matches a limit order, generate an attestation for a swap, including the order and the amount
5. Aggregator collects Attestations from all Vectrs. Once over 2/3 has signed an *identical* attestation, batch them in a transaction.
6. Submit transaction to Verification Module which ensure sufficient Vectrs have attested as defined in the Security Module
7. Once verified, Verification Module calls Limit Order contract to trade a given position.
8. The users who depositred in the Limit Order contract can withdraw their swapped tokens at any time, knowing they got their desired price. They could also withdraw the original tokens by cancelling the limit order if it is never executed.

Value Proposition: Managing a large orderbook for limit orders can quickly become computationally infeasible to run for every swap and process on a blockchain itself. By doing all the matching off-chain in a highly parallelizable manner off-chain we remove that bottleneck. The tokens themselves remain secured on-chain and all swpa rules can be enforced their as well (in case their was a quick price change between the detection and the swap execution).

Flexible Oracles

Flexible oracles, return time weighted average price, using the mean between multiple Vectrs and punishing extreme outliers. This power and ease of deployment can easily be tailored for prediction markets, perpetual futures and other on-chain use cases that need a trustworthy oracle solution. It includes:

- Cron Events
- Web2 queries
- Local Storage
- Composition Events (multi-round circuits)
- On-chain verification

This is described in more detail in Implementation Milestone 4.

User Onboarding

With WarpDrive, you can allow on-chain actions to be securely controlled by off-chain events. This opens up many possibilities for increasing the ease of use of Stellar, rather than requiring to sign every action with a wallet. Ethereum has long been pushing ease-of-use to increase onboarding via things like Account Abstraction and WalletConnect. With WarpDrive you can take this to the next level. In fact, we have previously built some of these with WAVS, each one taking no more than a week or two to build:

Sending tokens via Telegram: You grant access on-chain one time, proving a smart contract with some tokens and the Telegram username that can control them. Later, you can simply send a Telegram message from that username to a bot account, and WarpDrive will verify it and send the tokens to whoever was specified in the message. You can also register your Telegram handle so people can send directly to your username and you will receive them in your Stellar account. Built for Cosmos Hackathon

Email Controlled Accounts: This is a clever approach to enable on-boarding from exchanges. A user could deposit tokens directly from an exchange into an eg. lending protocol-controlled smart contract on Soroban without ever having a Soroban wallet. But when they create this deposit, they assign an email that can control the deposit. Later they can send an email to withdraw their tokens (with interest) back to the exchange, they send an email to a protocol-controlled address. The protocol shares this email along with all DKIM headers and signatures with the Vectrs, who can cryptographically validate that it was properly signed by the registered sender, and then take any on-chain actions - like withdrawing from the lending protocol, swapping rewards for USDC, and sending all value to an exchange-controlled wallet. They can earn on Soroban without ever installing a wallet. This has been built for Hydro, a Cosmos / Ethereum project

BlueSky Posts: What if you want to authenticate BlueSky posts and releasde some rewards when your project has gotten 500 retweets? Or even allow sending tokens or withdrawing shares via a BlueSky post (like the Telegram and email examples above)? We support streaming posts from AT protocol and can use those to securely trigger any action via the decentralized Vectrs, just as if the event originated with Stellar or Ethereum. There is a working proof-of-concept that was used to make Ethereum attestations for HyperCerts.

These are just the tip of the iceberg and what has been built so far. You could imagine Oauth signing to allow account recovery via "Login with Google" or Apple once you set this up on-chain. Any web2 event that can be checked can be used to control on-chain actions.

Implementation Roadmap

1. Detailed design and infrastructure Setup

This milestone is about validating all questions and producing contract interfaces and setting up needed infrastructure to run the system.

- Create detailed Soroban interfaces for all contracts in Milestone 2 (empty implementations)
- Test how many secp256k1 and ed25519 signatures we can fit in one transaction and any optimizations needed
- Designing cross-contract authorization patterns to understand how to map our validation patterns to Soroban's auth system
- Get a contract with an infrastructure provide to stream and query block events, and ensure we get timely event streams
- Test if we can parse events fully with Soroban.RS or need other libraries
- Test if Soroban.RS can compile to WASI and map out any changes needed

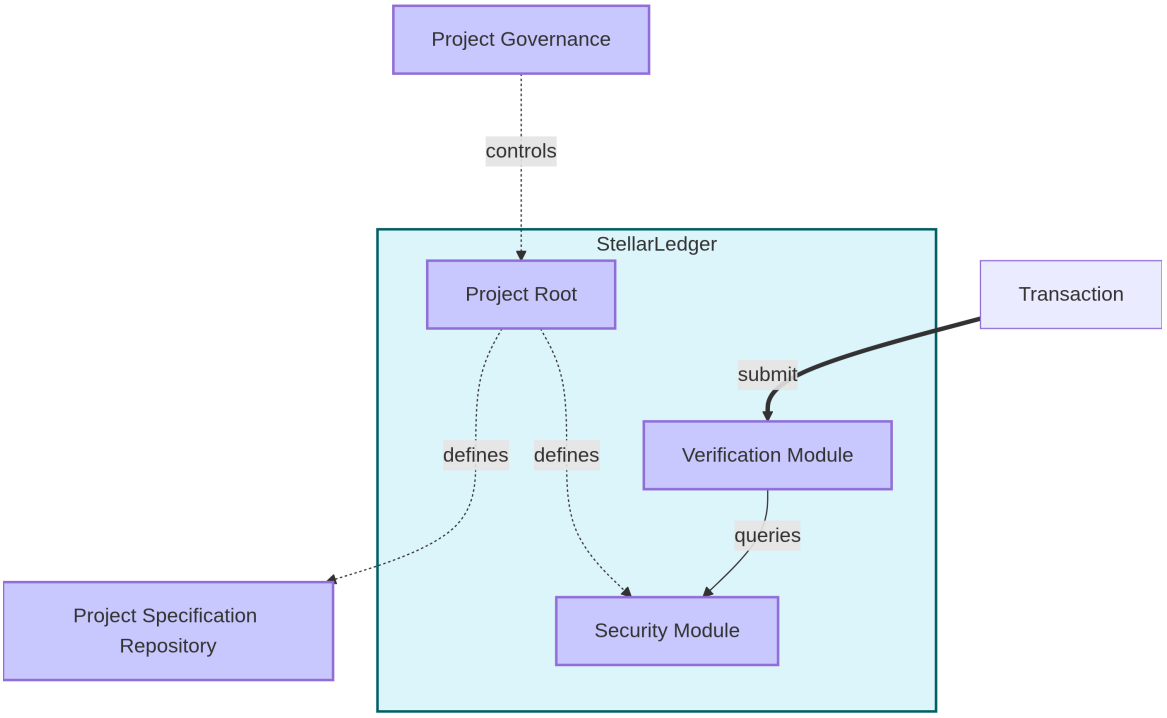
Deliverable: We have detailed documentation for starting both the contract work and the off-chain work. We have benchmarked sample signature verification code and found any limits on Soroban we may need to adapt to. We have set up an RPC endpoint for all development and a simple rust script interacting with Soroban via the RPC. We have tested various off-chain Rust libraries and settled on the proper setup here, noting any development work that would be needed to integrate with current Vectrs setup.

2. Soroban Security Contracts (PoA)

- Project Root Contract
- Security Contract (PoA)
- Verification Contract
- IPFS as Project Repository

In this milestone, we set up and test the core on-chain components that can be used to securely settle off-chain AVS systems.

Deliverable: Create a custom repo `wrap-drive-contracts` that contains all the needed contracts, the deployment scripts, as well as scripts for deploying the Project Specification Repository to IPFS. We will also provide automated tests to generate and aggregate multiple attestations off-chain and submit them to the verification module, which can use the security module to enforce an N of M multi-sig like rules. We will support both `ed25519` and `secp256k1` signatures inside the transaction payload and verify them with Soroban host functions to enforce any authorization rules inside the contract itself. `ed25519` is supported for compatibility with Stellar infrastructure and to allow these contracts to be part of generic multi-sig infrastructure. But we will focus on `secp256k1` validation as the off-chain infrastructure that is compatible with Ethereum and Cosmos currently uses `secp256k1` inside the payloads.

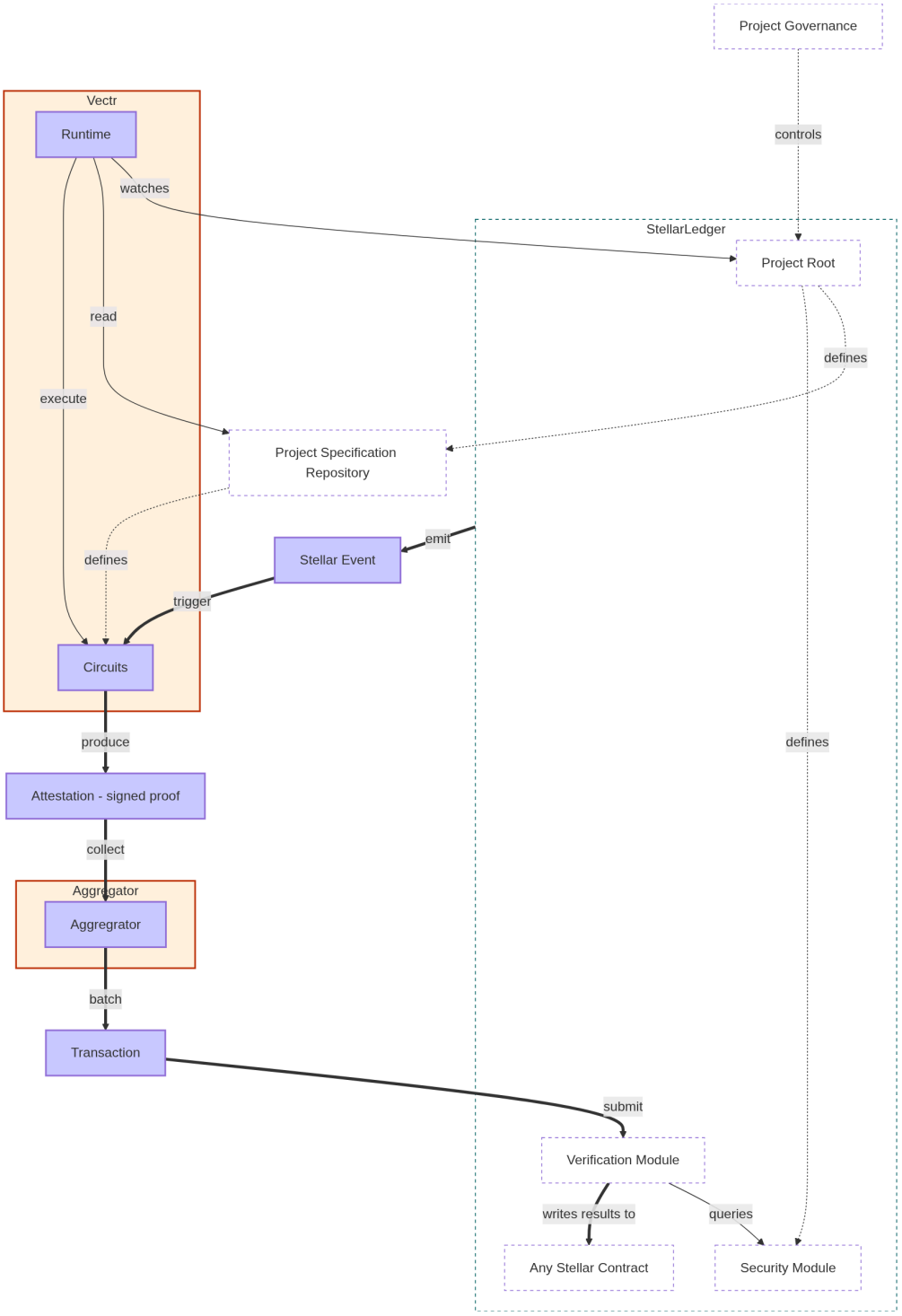


3. Soroban-Vectr Integration: DeFi Automation

- Port Vectr from WAVS
- Query project information from Soroban Project Contract
- Support listening for Soroban events in Circuits
- Support aggregator posting transaction to Soroban Verification Contract
- Ensure circuit configuration and running

In this milestone, we make all changes to existing WAVS off-chain runtime needed to support Stellar. While the basic infrastructure for Runtime, Circuits, and Aggregator existins, we need to extend it to support listening to Stellar and Soroban Events, as well as submitting valid Soroban transactions with the results. Extending these also involves extending the Circuit schema, and modifying the core runtime to be compatible with our new Project Root. By fuflilling this deliverable we will have the MVP of a complete end-to-end WarpDrive workflow involving a set of Vectrs that are processing Soroban events and submitting them on-chain with proper validation.

Deliverable: To provide an example of off-chain automation that can securely and non-invasively extend a DeFi protocol, we will create a new contract called Hodlers, which maintains "points" for a set of Stellar addresses. We will then build a simple Circuit that listens for any trades on the XLM-USDC pool on Phoenix. With each swap, the circuit will submit a transaction to the verification contract, which has admin access to Hodlers. It will then add an amount of "points" to the swappers address if it bought XLM, and reduce the points if it sold XLM. Hodlers is just counting the trades, but it could take other actions (like provide rewards), and shows it is possible to securely and verifiably add arbitrary DeFi automation to Stellar.


Project Governance
controls
Vectr
Runtime
watches
StellarLedger
Project Root
read
defines
execute
Project Specification Repository
emit
Stellar Event
defines
trigger
Circuits
produce
defines
Attestation - signed proof
collect
Aggregator
Aggregrator
batch
Transaction
submit
Verification Module
writes results to
queries
Any Stellar Contract
Security Module

4. Real World Usage

This is actually shipping real-world examples on WarpDrive. There are three main sections here:

- Price Oracle demo (running on mainnet Soroban, but not connected to protocols)
- A real-world DeFi automation protocol live on mainnet with real liquidity
- Project Starter Pack to launch new WarpDrive projects

We willl finalize this grant by bringing all the pieces together and providing tangible use cases. This inclues both a generic and specific example running on mainnet to inspire users, as well as immediately adding more DeFi activity to Soroban. And we also provide a clean path for other projects to add WarpDrive-powered functionality to their protocols. Let's look at the sub sections:

Complex Workflows: Price Oracle with median TWAP from exchanges

- Test and optimize all Soroban integrations
- Demonstrate and test reacting to Ethereum events
- Demonstrate and test reacting to Web2 events
- Demonstrate and test multi-circuit composition
- Project Starter Pack to launch basic project easily

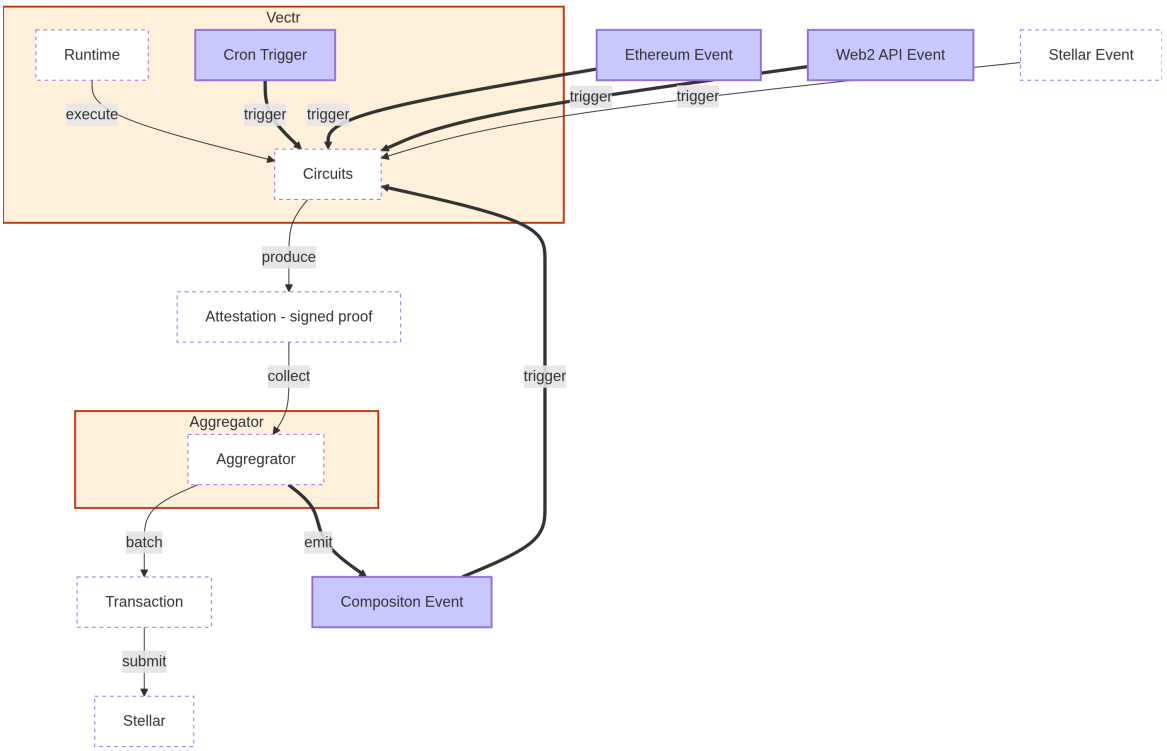
In order to create this demo, we must extend the full functionality of Warp Drive triggers, adding and testing various other mechanisms for triggering circuits. This includes Cron driver triggers, but also reacting to Ethereum events, and Web2 Events (ex. Bluesky posts). Finally, we will enable feedback via Composition Events, where the aggregator can submit the output of one circuit back into the Vectrs, one sufficient signatures have been collected, in order to trigger a second circuit for multi-step processing.

Deliverable: A multi-round price oracle providing secure, time-weighted CoinGecko prices for BTC-USD and ETH-USD. Each Vectr will run multiple circuits:

1. A `Cron Event` to check the CoinGecko prices every 10 minutes and store it locally
2. A `Stellar Event` from an event of an OracleContract, including the pair and the time range (1hr, 6hr, 1 day) aslong with a request id. This will trigger all Vectrs to calculate a Geometric TWAP (geometric mean) over their locally stored values in that time range and attest to it.
3. The aggregator will wait for 80%+ of the Vectrs to return results, then package all their attestations into a `Composition Event`. Each Vectr will detect this on another circuit, validate that all the signed attestations belong to other Vectrs from this project (query Security Module and validate signatures), and then calculate a median price over this data.

This final median TWAP will be submitted to the verification module on Soroban, which will store the result in the Oracle Contract if all attestations are validated. This shows an important case of two-round off-chain computations, which often involves on-chain processing in other Oracle frameworks (commit and reveal). The reason is that due to changing spot prices, two correct Vectrs may have slightly different price data from Coin Gecko (usually within 0.5% but still not *exact*). Normal attestations require *exact* matches, which works when responding to blockchain events (as in Milestone 2), but not from many web2 APIs which may produce slightly different results if queried a few seconds apart from different machines.

This deliverable shows Warp Drive can handle such complex workflows and produces a useful and valuable sample app that could easily be extended to support many powerful DeFi use cases on Soroban.



Real-World Deployment: DeFi Automation

We have discussed with Phoenix and Blend protocols to deploy an automation protocol to connect their respective protocols in a synergetic manner. This will involve placing 50% of the USDC in the XLM-USDC pool in a Blend lending protocol to collect interest. As long as 50% is liquid, all trades will go through with no problem, and the Warp Drive circuit will actively monitor the pool size, moving tokens between the Blend lending poll and the LP liquid balance as needed to maintain this 50% ratio. The Warp Drive circuit will also collect the USDC Rewards from Blend which will be paid out to all LP holders (by depositing them into the pool without taking LP tokens).

This concept has been implemented and tested on mainnet Ethereum via Balancer Boosted Pools, but it took this top-tier team almost two years to roll it out. By leveraging WarpDrive's unique technology, we will demonstrate that such cutting edge features can be rolled out quickly and safely, with minimal changes needed to on-chain contracts.

Phoenix will respect their token holders by creating a new XLM-USDC pool with this feature enabled, and allow existing LP holders to easily migrate their positions from the current pool to the new pool. The frontend will handle routing between the two pools for best rates, and the team will migrate over their incentives to the new pool to encourage adoption.

While the Warp Drive circuit is not much more complex that milestone 2, this will involve a lot of real integrations on the Soroban blockchain, and run on a properly distributed operator set, providing us the first real-world deployment managing significant value. As much of this work will be done by the Phoenix team, and few of the features of milestone 3 are required for the execution of this project, milestones 3 and 4 can be pursued concurrently.

- Finalize design of the integration including agreement by both core teams
- Build out the custom verification contracts to integrate the two protocols (invest liquid LP tokens in blend pools)
- Build out off-chain circuits and logic to handle rebalancing positions efficiently
- On-board multiple operators to run Vectrs
- Deploy Project and Circuits to live Vectrs
- Phoenix adds new pools with the Blend integrations, users can migrate over

- Provide web UI (dashboard) to show the function of the Warp Drive circuits (investments and rebalancing)

Project Starter Pack

The final focus of this milestone is ensuring all the aspects are robust and easy to deploy. This includes more automated integration tests, but also developer tooling and documentation. We will produce a well-documented project that can be used as a starting point for developers, demonstrating many of the aspects of the system and including deployment instructions.

We will also create a public chat for any projects interested in WarpDrive, to provide support as well as connect them with operators excited about running WarpDrive Vectrs and securing their protocol. This is really the part where WarpDrive can take off in the Stellar and Soroban ecosystems, and where the long-term value to Soroban developers comes in.

Operational Flow

Setup

Developmenmt

Projects start with local development:

- Write execution logic for verification module
- Write any external contracts needed to be called by verification modules
- Write WASI components to process data in the circuits (their output is fed to the verification module)
- Define all circuits in a JSON file

Initial Deploy

Then we deploy:

- Setup Project Governance (Stellar multisig)
- Deploy Project Root, Verification Module, an empty Security Module, and any external contracts needed for integration
- Generate project specification, include pointer to the deployed Project Root and Security Module, and upload to IPFS
- Project Governance updates Project Root to point to the proper Project Specification
- Multiple operators start up the standard WarpDrive executable on their Vectrs and register this Project Root.
    - Their WarpDrive software will start running the project circuits on their Vectrs
    - Operators get a new key pair to be used with this project, and register the public key on the project
- Project Goverance collects the public keys of all trusted Vectrs and then configures the Security Module with those keys
- DONE: All Vectrs running the circuits, and creating valid attestations that can be confirmed by the Security module

Note: that if an operator wants to run multiple projects on one Vectr (and has sufficient compute power for it), they can easily run multiple projects on the same WarpDrive software, sandboxed from each other. This makes deployment of multiple projects easier from an operational perspective (no need to set up more physical machines).

Upgrading Circuits

- Developers write new circuits and WASI components
- Developers deploy any needed on-chain contracts to handle new circuits
- Upload all components and configuration to the Project Specification Repository
- Project Governance updates Project Root to point to this new Specification
- All Vectrs detect this chain, and update their local circuits to match the new specification
- DONE: All data flows through the new updated circuits

Component Deep-Drive

The diagrams above show the logical flow. This section explains what each component actually does and why the architecture makes these choices.

1. Governance & Specification Layer

Sub-component	Purpose	Interaction points
Projects	Logical sandbox for a single business use-case.	The Project contract is the source of truth that Vectrs query for the spec, and that the Verification Module consults to enforce the correct Security Module.
Project Governance	Permisisoning to update a given Project	This account or contract is the admin and can update the Project Contract
Project Specification Repository	Immutable, off-chain source of truth for a Project's circuits and policies.	Vectrs pull all confiuguration and executable code from this repo as specified as the Project contract, which defines their activity.

Projects

Every deployment starts with a Project contract on Soroban. This contract stores the current specification hash and controls all parameters—verification rules, security policies, and circuit configurations. Each Project operates in complete isolation from others; they share no state and interact only through standard Soroban contract calls. This isolation means a vulnerability in one Project cannot compromise another, and each team maintains full sovereignty over their deployment.

Project Governance

The governance address holds the keys to the kingdom. Only this address can update the Project contract, making its security critical. Warp Drive supports any governance model Soroban can express: a native multisig for enterprise deployments requiring board approval, a DAO contract like Soroban Governor for community-driven projects, or a single key for rapid prototyping. The architecture imposes no constraints—teams choose the trust model that fits their operational reality.

Project Specification Repository

The specification repository holds everything Vectrs need to execute: JSON files describing each circuit's trigger conditions, WASI payloads, and output targets, plus the compiled bytecode of every WASI component (referenced by SHA256 hash for integrity verification). It also stores addresses of all Soroban contracts in the Project, including a back-reference to the Project Root that defines it. Content-addressable storage (IPFS or similar) makes this practical—the on-chain contract stores a single hash, while the full specification remains immutable and globally accessible.

This separation is deliberate. Governance lives on-chain where it benefits from Stellar's security guarantees. Heavy artifacts live off-chain where storage is cheap and retrieval is fast. When a team needs to upgrade their circuits, they push a new specification and update one hash on-chain. Every Vectr detects the change and pulls the new code automatically. Upgrades that would require contract migrations on other platforms happen in seconds on Warp Drive—with a complete audit trail preserved in the ledger.

2. Execution Layer

Sub-component	Purpose	Interaction points
Verifiable Computers (Vectrs)	Decentralised compute pool that runs circuits.	Vectrs receive triggers directly from the Stellar network (or from configured web2 webhooks). After execution they submit the attestation to the **Verification Module** contract.
Circuits	Formal description of an end-to-end off-chain job.	The spec file lists circuits; Vectrs read this list, match incoming triggers to the proper circuit, and know exactly which output contract method to invoke after verification.
Composable Workflows	Enables chaining of circuits across Projects.	Workflows are expressed purely in the Project Specification; no extra on-chain glue is required. The Verification Module treats each circuit independently, while the off-chain orchestration (via Vectrs) handles the chaining.

Verifiable Computers

Vectrs are the workhorses of Warp Drive—independent execution nodes that run circuits and produce cryptographic attestations. On startup, a Vectr queries the Project Root contract to find the current specification, pulls the circuit definitions and WASI binaries from the repository, and subscribes to the Project Root for automatic updates. When the specification changes, Vectrs detect it immediately and reconfigure themselves without operator intervention.

Once running, a Vectr listens for triggers across all configured circuits. When a trigger fires—an on-chain event, a cron tick, an API callback—the Vectr executes the corresponding WASI payload inside a sandboxed wasmtime runtime. The output becomes a signed attestation. Critically, the signing key lives outside the sandbox; the WASI code cannot access it, cannot exfiltrate it, and cannot forge signatures. Each operator generates a unique keypair per project, ensuring that compromise of one project's key reveals nothing about others.

Circuits

A circuit defines a complete unit of off-chain work through three components. The **input** specifies what triggers execution: an on-chain event from Stellar or Ethereum, a cron schedule, or an external API subscription. The Vectr's monitoring engine handles all three with equal efficiency. The **transform** is the WASI component itself—compiled from Rust, Go, or JavaScript—that processes the trigger. This code can make network calls, query blockchains, read and write local storage, and perform arbitrary computation. The **output** specifies where attestations go once enough Vectrs have signed: typically a Soroban verification contract, but the architecture also supports EVM chains, IPFS for public records, or feedback into another circuit for multi-stage processing.

Composable Workflows

Real business logic rarely fits in a single step. A price oracle might aggregate data, compute a median, then post to multiple chains. A compliance engine might check a transaction, query external KYC providers, and conditionally approve or flag. Warp Drive handles these through composable workflows: one circuit's output becomes another circuit's input, without touching the blockchain between stages.

This composition happens entirely off-chain but retains full security guarantees. Each Vectr verifies the signatures on incoming attestations using the on-chain Security Module as the source of truth. If sufficient weight has signed, the Vectr treats the attestation as authoritative and proceeds to the next stage. Only the final result settles on-chain.

The practical benefits are substantial. Multi-stage computations maintain the same security level as simple circuits—no trust degradation as complexity grows. Intermediate steps skip blockchain settlement entirely,

eliminating gas costs and reducing latency from block times (seconds) to network round-trips (milliseconds). Complex workflows that would be prohibitively expensive or slow on-chain become practical. Warp Drive is not a single-function oracle service—it is a platform for building verifiable, multi-step business processes on Stellar.

3. Trust, Verification & Composability

Sub-component	Purpose	Interaction points
Verification Module	On-chain gatekeeper that validates attestation proofs.	Receives attestation from Vectrs, reads the Security Module policy from the Project contract, and finally triggers the destination contract defined by the circuit.
Security Module	Determines *who* can attest for a given Project and with what weight.	The Verification Module loads the policy at verification time; Vectrs embed their public key in the attestation, allowing the contract to look up the allowed weight.

Verification Module

The Verification Module is the on-chain gatekeeper—the contract that decides whether off-chain work becomes on-chain reality. When attestations arrive from the aggregator, the Verification Module passes them to the Security Module for signature validation. If the signatures carry sufficient weight, the module parses the payload and acts on it.

The design is deliberately minimal. The Verification Module handles attestation intake, delegates cryptographic validation to the Security Module, and translates verified payloads into contract calls. This separation keeps each component focused and auditable. Developers writing custom Verification Modules need only implement the application-specific parsing and action logic—the security guarantees come from the shared infrastructure.

Security Module

The Security Module answers one question: should this attestation be trusted? It maintains a registry of public keys with associated weights and exposes a verification function that checks whether a set of signatures meets the required threshold (typically two-thirds of total weight). The module is agnostic to payload contents—it validates signatures on bytes, nothing more.
This abstraction enables powerful flexibility in how trust is established. A project can configure the Security Module as a static Proof-of-Authority list: a fixed set of trusted Vectr public keys, controlled by governance. Alternatively, it can implement Proof-of-Stake with EigenLayer-style restaking, where operator weights derive from staked XLM and slashing rules enforce honest behavior. Or it can define any custom algorithm that maps public keys to weights—reputation scores, hardware attestations, or hybrid schemes combining multiple signals. In the future, we will add support for slashing as well here - it is the central location for all security-related concerns and can be reused wihtout changes by all Projects.

Conclusion

Stellar's strength is fast, low-cost, compliant transactions. Its limitation is the same as every blockchain: smart contracts cannot reach beyond the ledger. They cannot fetch external data, respond to real-world events, or coordinate complex multi-step processes without trusting a single operator.

Warp Drive eliminates this constraint.

By providing cryptographically verifiable off-chain compute, Warp Drive transforms Stellar from a settlement layer into a full execution platform. Oracles, automated market makers, cross-chain bridges, compliance engines, and AI-driven agents all become possible—without sacrificing the security guarantees that make blockchain valuable in the first place.

The architecture is production-ready. WASI execution runs at native speed. The security model supports both enterprise-grade Proof-of-Authority and permissionless Proof-of-Stake, letting each project choose its trust profile. The implementation roadmap delivers working code at every milestone, culminating in a live Phoenix-Blend integration that demonstrates real economic value: automated yield optimization across Stellar's DeFi ecosystem.

This is not speculative infrastructure. The core components—deterministic execution, threshold signatures, on-chain verification—are proven patterns. What Warp Drive adds is the engineering to make them work seamlessly on Stellar, with tooling that lets any developer deploy verifiable off-chain logic in hours rather than months.

Funding Warp Drive means Stellar gets:

- **Immediate utility**: A working oracle and automation framework within the grant period.
- **Ecosystem growth**: Lower barriers for developers building sophisticated applications.
- **Competitive positioning**: Infrastructure parity with chains that currently dominate the verifiable compute narrative.
- **Real integration**: The Phoenix-Blend use case proves value to existing Stellar protocols and their users.

Warp Drive is ready to build. The architecture is sound. The team has delivered. Fund this grant, and Stellar gains the trusted compute layer that turns its speed and efficiency into a platform for the next generation of decentralized applications.

Addendum: Performance

High performance WASI, websocket requests, concurrent Rust processes... super high-throughput, limited by on-chain settlement.

WASI execution runs at near-native speeds and startup adds around **400 microsecond overhead**. We can also run one execution thread per core of the Vectr machine, so will run as fast as the hardware allows. The only pieces that would slow down are network requests and large computations (like runningh an LLM locally, or performing homomorphic encryption). Everything will run faster than your laptop and concurrently - far beyond blockchain levels.

We listen to events via websockets (and gRPC streams for some web2 APIs) providing minimal latency between the event occurring and the data being processed by the WASI model.

Signing the attestation takes around **20 microseconds** for ed25519 curves (using ed25519-dalek) and around **100 microseconds** for secp256k1 curves.

We have benchmarked throughputs of over 10,000 events per second, which overwhelms the ability of the blockchain to process them. So users should feel free to place arbitrary computation offchain without any concern about sufficient latency or throughput of the Warp Drive system.

Addendum: Governance & Trust Model

Warp Drive must guarantee that the off-chain **Vectrs** produce *authentic* attestations and that any mis-behaviour can be punished and proven on-chain. Two well-understood paradigms satisfy this requirement:

Model	Core idea	How trust is established	Reward / penalty mechanism	Typical on-chain evidence
Proof-of-Authority (PoA)	A small, pre-selected set of operators is granted the right to sign attestations.	Authority is granted by the **Project's governance contract** (often a multisig or DAO). The list of authorized public keys is stored on-chain and can only be changed by a governance vote.	Operators receive a fixed fee per successful attestation (paid by the Project treasury). Mis-behaviour (e.g., double-signing, missing deadlines) is detected by the Verification Module; the offending key can be *removed* by a governance proposal and the operator forfeits any pending fee.	• Signature on the attestation matches a key in the on-chain whitelist. • Governance proposal transaction that adds/removes keys. • Event logs showing slash/freeze of the operator's fee escrow.
Proof-of-Stake / Delegated PoS (dPoS) Restaking	Operators earn the right to attest by staking (or being delegated) a	Trust is proportional to the amount of stake backing an operator. The Verification Module reads the stake distribution from a	Operators earn a share of the Project's fees proportional to their stake weight. If an operator submits an	• On-chain proof that the operator's public key holds ≥ X tokens in the

Model	Core idea	How trust is established	Reward / penalty mechanism	Typical on-chain evidence
	token that is *restaked* on the Verification Module.	**Stellar-compatible staking contract** (or a bridge to Eigenlayer-style restaking).	invalid attestation, a slashing transaction reduces the staked amount (or a portion of the delegated tokens) and the loss is distributed to honest delegators.	staking contract at the time of attestation. • Slashing transaction that burns or transfers a fraction of that stake. • Delegation records that allow delegators to claim a portion of the slashed amount.

1. PoA – Strengths, Weaknesses & Attack Surface

Pros

1. **Fast onboarding & low latency** – No need to wait for stake bonding; a new operator can be added instantly by a governance vote.
2. **Predictable economics** – Fixed fees make cost modeling easy for enterprises; no market-driven fee spikes.
3. **Simpler client software** – Vectrs only need to hold a private key; no staking logic, no need to monitor slashing conditions.
4. **Clear accountability** – The whitelist is a small, auditable list; any mis-behaviour can be publicly blamed and the operator can be expelled quickly.

Cons

1. **Centralisation risk** – If the whitelist is static, the system collapses to a single-point-of-failure: a compromised operator or a colluding subset can produce fraudulent attestations.
2. **Governance bottleneck** – Adding/removing operators requires a governance transaction, which may be slow or contentious.
3. **Limited economic security** – Operators are only penalised by loss of reputation and possible removal; there is no *bond* that can be seized automatically.

Attack vectors

Vector	Description	Mitigation
Key compromise	An adversary obtains an authorized private key and signs false attestations.	Enforce multi-signature requirements (e.g., 2-of-N) inside the Verification Module; rotate keys regularly via governance.

Vector	Description	Mitigation
Insider collusion	A majority of the whitelist conspires to approve bogus outputs.	Require a quorum threshold (e.g., ≥ 2/3 of total weight) and optionally a *time-lock* before a result becomes final, giving challengers a window to submit a counter-attestation.
Governance capture	An attacker acquires enough voting power in the project's DAO to add malicious operators.	Use a *time-weighted* governance model (e.g., quadratic voting, delay on operator addition) and keep the operator set small enough that each addition is scrutinised off-chain.

2. dPoS / Restaking – Strengths, Weaknesses & Attack Surface

Pros

1. **Economic security** – Operators have *skin in the game*: any malicious attestation results in an automatic loss of staked tokens, which is verifiable and enforceable by the contract.
2. **Scalable validator set** – Anyone can become a Vectr by bonding stake, enabling a large, diverse pool that raises the cost of a 51 % attack.
3. **Dynamic trust** – Stake can be re-delegated at any time, allowing projects to shift weight toward well-performing Vectrs without on-chain governance changes.
4. **Alignment of incentives** – Rewards are proportional to stake, encouraging operators to keep their nodes reliable and honest to avoid slashing.

Cons

1. **Complexity** – Requires a staking contract, delegation UI, and slashing logic; this adds code-size and audit surface.
2. **Economic volatility** – Fees earned depend on the market price of the staked token; enterprises must manage exposure to token price swings.
3. **Potential for "rich-get-richer" centralisation** – Large delegators can concentrate voting power, which may lead to oligopolistic operator sets if not mitigated.

Attack vectors

Vector	Description	Mitigation
Stake grinding	An attacker buys a large amount of the staking token, becomes a top Vectr, and attempts to submit invalid attestations.	Set a *minimum* slashing penalty that outweighs the potential gain; use *bonded* stake that can be seized fully on mis-behaviour.
Delegator collusion	Delegators collectively vote to unbond honest operators and replace them with malicious ones.	Implement *delegation caps* per address and *time-weighted* delegation (e.g., lock-up periods) to slow rapid shifts.
Slashing denial-of-service	An attacker spams false attestation proofs, causing honest operators to be falsely flagged for slashing.	Require a *challenge period* where a counter-proof can be submitted; only slash after a deterministic verification that the proof is invalid.

3. Comparative Security – Why Both Models Beat a Central Server

Aspect	Centralised server (single owner)	PoA	dPoS / Restaking
Single point of failure	Yes – compromise or shutdown stops the whole service.	Reduced – failure of one operator only removes its weight; quorum still required.	Strongly reduced – attacker must control > 50 % of total stake to subvert the system.
On-chain auditability	No – logs live off-chain, tamperable.	Yes – the whitelist and any changes are stored in immutable contracts; each attestation is signed and can be verified.	Yes – stake balances, delegation, and slashing events are all on-chain, providing provable economic penalties.
Economic deterrence	None – only reputation or legal recourse.	Limited – removal from whitelist, loss of future fees.	High – automatic loss of bonded tokens, potentially confiscated by the project treasury.
Governance flexibility	Owner can change code unilaterally.	Operators can be added/removed by a vote; the process is transparent.	Operators can be added simply by staking; the project can adjust required quorum without contract upgrades.
Attack cost	Low – just compromise the host.	Medium – acquire a private key or gain governance control.	High – purchase enough stake to influence the set *and* risk slashing.

In practice, **Warp Drive can expose both models** through the **Security Module** configuration:

- A project that values *predictable latency* and *strict corporate oversight* may select PoA with a multi-sig quorum (e.g., 3-of-5 authorized Vectrs).
- A project that wants *open participation* and *strong economic guarantees* can enable the dPoS/Eigenlayer-style restaking path, specifying minimum stake, slashing percentages, and delegation caps.

Both approaches are **orders of magnitude more secure** than a traditional centralized server:

- The attack surface is distributed across many independent nodes, raising the cost of a successful breach.
- All critical state transitions (operator set, attestation acceptance, slashing) are recorded on the Stellar ledger, making tampering detectable and reversible.
- Economic incentives are enforced automatically by smart contracts, removing reliance on off-chain legal enforcement.

TL;DR

- **PoA** offers fast, low-complexity governance with clear accountability, but must rely on a robust on-chain voting process to avoid centralisation.
- **dPoS / Restaking** provides strong economic security and dynamic participation at the cost of added protocol complexity.
- Warp Drive's **Security Module** can be configured for either model—or even a hybrid (e.g., PoA whitelist that must also hold a minimum stake), giving projects the flexibility to choose the trust level that matches their risk profile while guaranteeing far-greater security than any single-owner server architecture.